Faculty of Information Technology and Communications
New Cairo University

ICT-N-322 / ICT-D-324 · Network Programming

ChatNet

Network Programming Project

A Comprehensive Multi-Protocol Chat System

Submitted in partial fulfillment of the requirements for ICT-N-322 / ICT-D-324

Section 1 — Planning & Modelling

1.1 Requirements Analysis

ChatNet is a hybrid networking system designed to demonstrate practical mastery of Application and Transport layer protocols. The system implements all five key networking services from raw Python sockets, without the use of third-party libraries or high-level frameworks.

Design Principle: All protocols — TCP chat, UDP file transfer, DNS resolution, HTTP log serving, and SMTP notification — are implemented manually using only the Python socket module and threading.

Component	Protocol	Purpose	LO
TCP Chat Core	TCP / SOCK_STREAM	Reliable ordered message delivery between clients	LO2, LO3
UDP File Transfer	UDP / SOCK_DGRAM	Low-overhead chunked file transfer (Stop-and-Wait ARQ)	LO3
DNS Resolver	UDP to 8.8.8.8:53	Manual binary DNS query and A-record parsing	LO4
HTTP Log Server	TCP / HTTP 1.1	Serve last 50 chat messages as HTML on port 8080	LO4
SMTP Notifier	TCP / SMTP+STARTTLS	Email alert on @mention; raw socket handshake	LO4
Diagnostics	TCP (internal)	/ping RTT measurement; /throughput kbps analysis	LO2

1.2 Use Case Diagram

The diagram below shows the three primary actors interacting with ChatNet and the actions available to each.

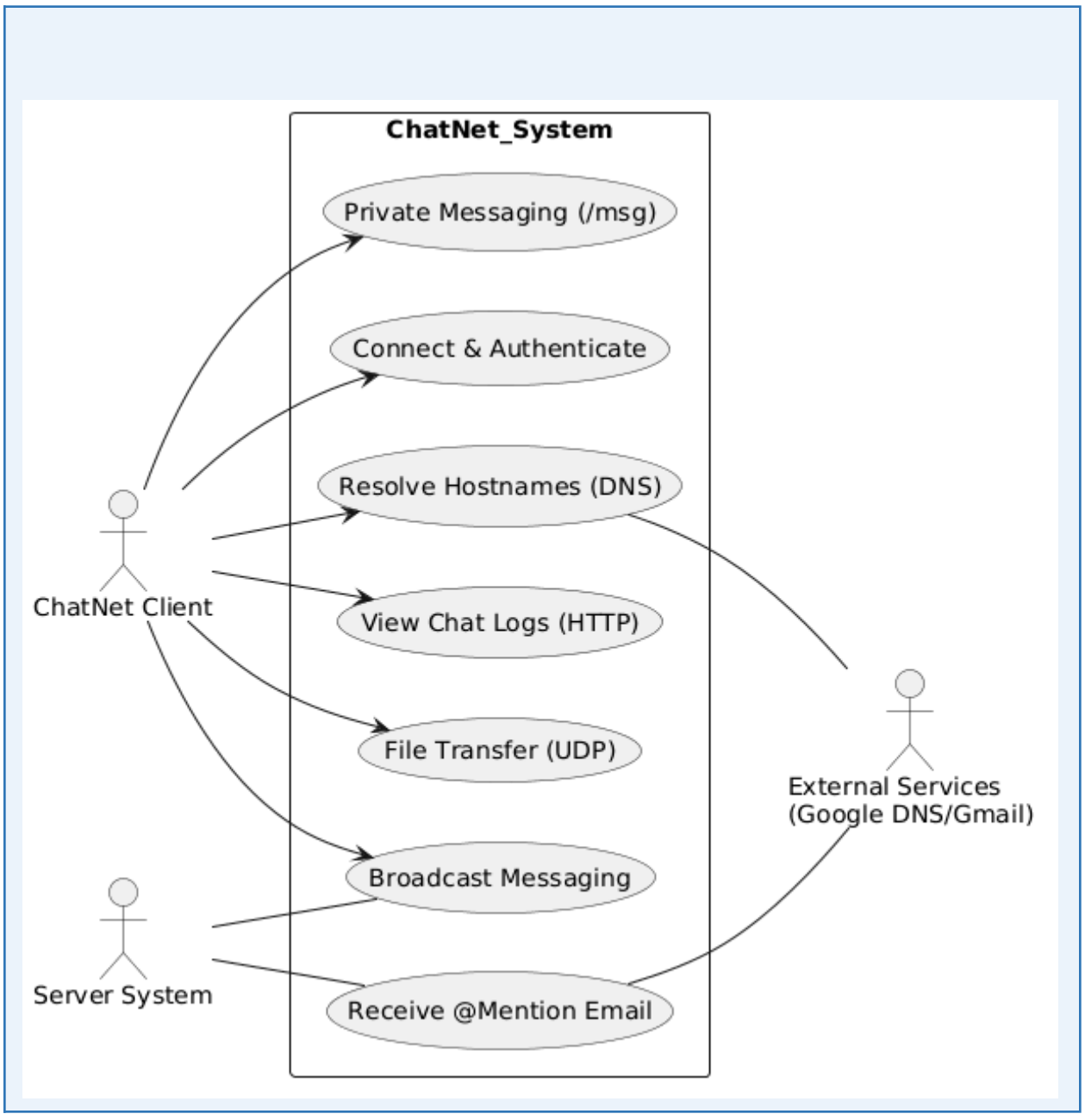


Figure 1.1 — ChatNet Use Case Diagram

1.3 Sequence Diagrams — Key Communication Flows

A. TCP Connection Setup & Username Exchange

Illustrates the three-way TCP handshake, the server's 200 OK welcome message, and the username negotiation phase. The server validates uniqueness and rejects duplicates with 409 Conflict.

Step	Direction	Message / Action
1	Client → Server	TCP SYN (Transport layer handshake)
2	Server → Client	TCP SYN-ACK
3	Client → Server	TCP ACK (Connection established)
4	Server → Client	"200 OK ; Connected to ChatNet Server"
5	Server → Client	"Enter Username:"
6	Client → Server	"Ahmed" (username string)
7	Server (internal)	Validate: username unique? → else 409 Conflict
8	Server → Client	"Welcome Ahmed!"

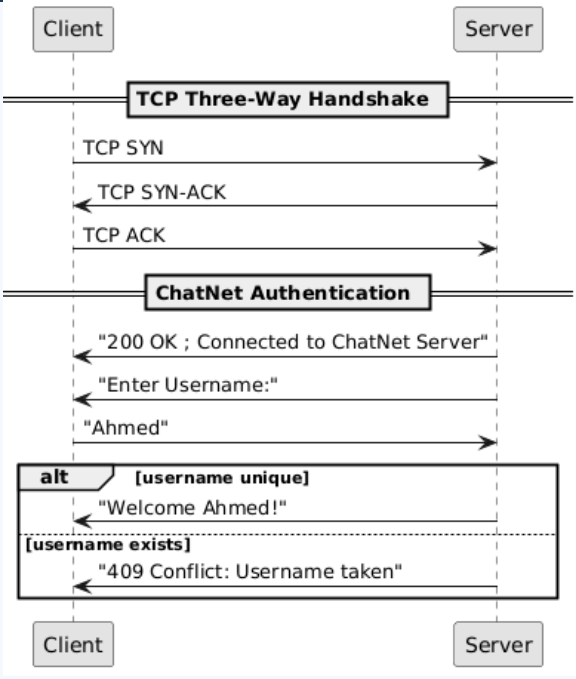


Figure 1.2 — TCP Handshake Sequence Diagram

B. Message Broadcast

When any client sends a message the server formats it with a server-side timestamp and broadcasts to all connected sockets via the protected clients dictionary.

Step	Direction	Message / Action
1	Client A → Server	TCP MSG: "Hello"
2	Server (internal)	Acquire threading.Lock() on clients dict
3	Server (internal)	Format: "[HH:MM:SS] Ahmed: Hello"
4	Server → All clients	Broadcast formatted message to every socket
5	Server (internal)	Release Lock()

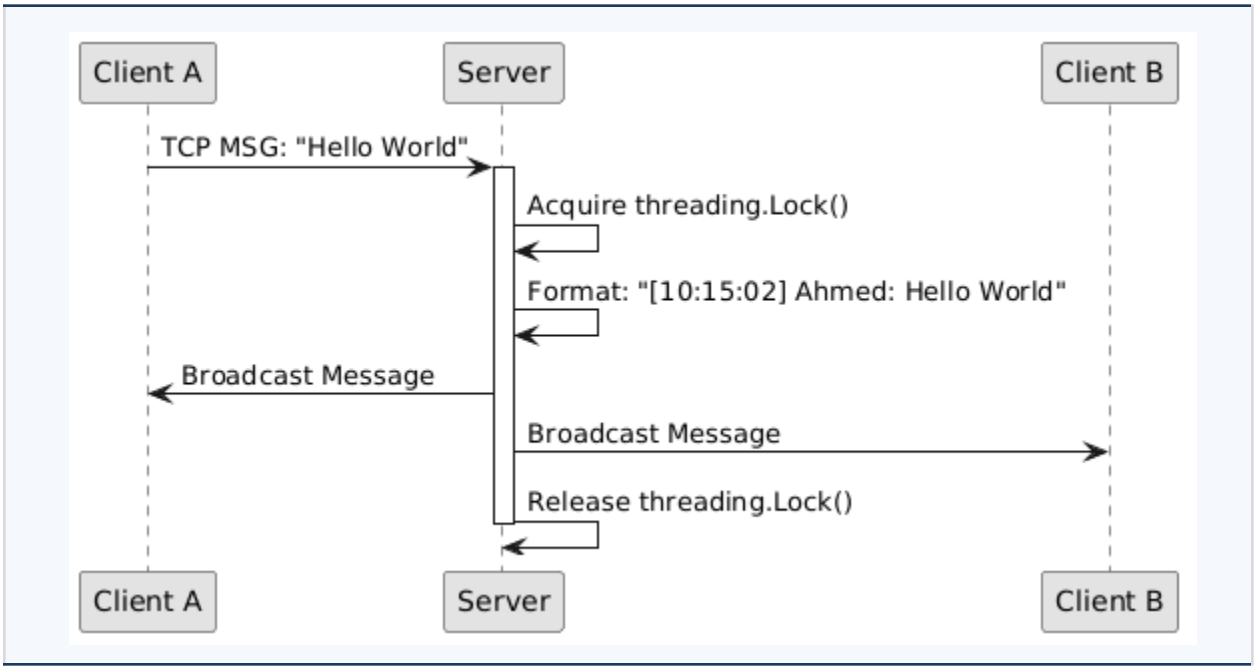


Figure 1.3 — Broadcast Sequence Diagram

C. UDP File Transfer — Stop-and-Wait ARQ

The sender divides the file into 512-byte chunks. Each chunk carries a 4-byte sequence number header. The sender waits for an ACK before sending the next chunk; a 2-second timeout triggers retransmission.

Step	Direction	UDP Packet Content
1	Sender → Receiver	FILEHEADER { filename, total_chunks }
2	Receiver → Sender	ACK_HEADER
3	Sender → Receiver	Data Chunk 0 [seq:0 | 512 bytes]
4	Receiver → Sender	ACK 0
5	Sender → Receiver	Data Chunk 1 [seq:1 | 512 bytes]
6	Receiver → Sender	ACK 1
…	…	Repeat until all chunks acknowledged
N	Sender → Receiver	FILEEND marker
N+1	Receiver (internal)	Reassemble chunks → write file to disk

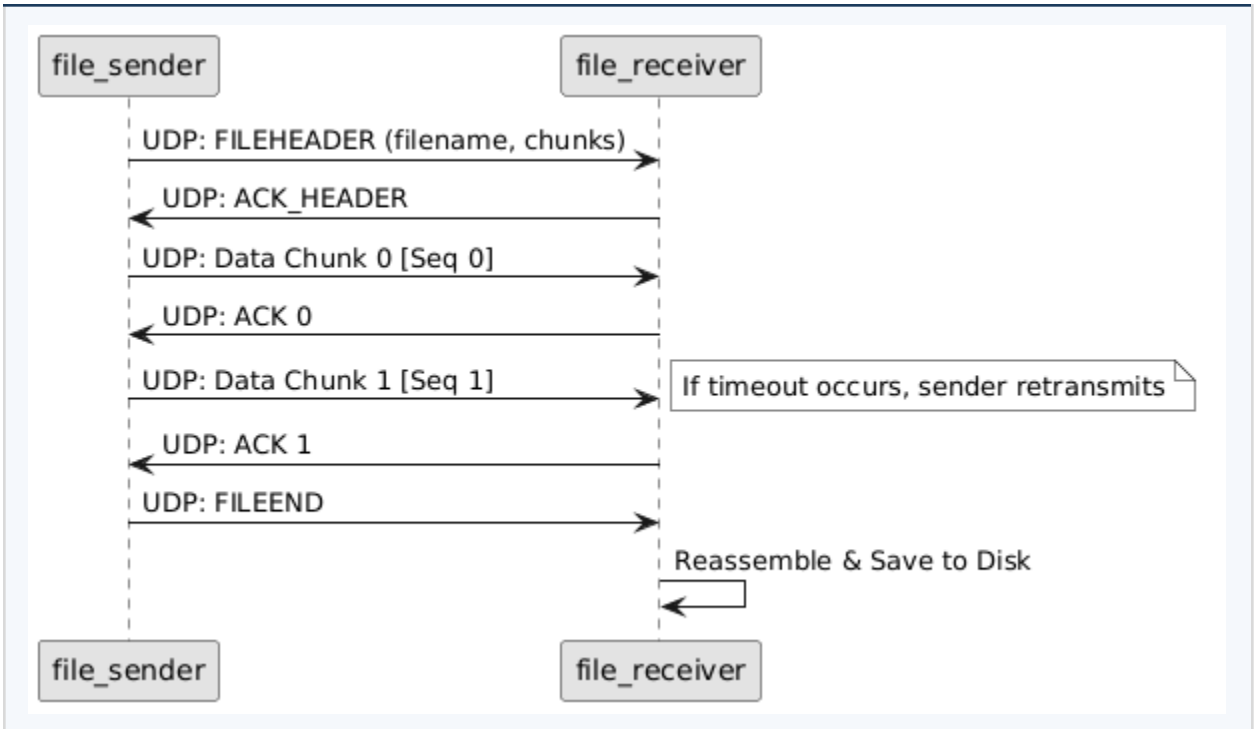


Figure 1.4 — UDP File Transfer Sequence Diagram

D. DNS Binary Query (Manual)

dns_resolver.py manually constructs a binary UDP packet conforming to RFC 1035, sends it to 8.8.8.8:53, and parses the response Answer section to extract the A record (IPv4 address).

Step	Direction	Action / Content
1	Client (internal)	Construct Binary Header: TxID | Flags | QDCOUNT=1 | ANCOUNT=0
2	Client (internal)	Construct Question: QNAME (label-encoded) | QTYPE=1 (A) | QCLASS=1 (IN)
3	Client → 8.8.8.8:53	UDP packet: Header + Question (SOCK_DGRAM)
4	8.8.8.8 → Client	UDP response: Header + Answer section
5	Client (internal)	Parse Answer section → extract RDATA (4-byte IPv4)
6	Client (output)	Print: "Resolved chatnet.local → 192.168.1.10"

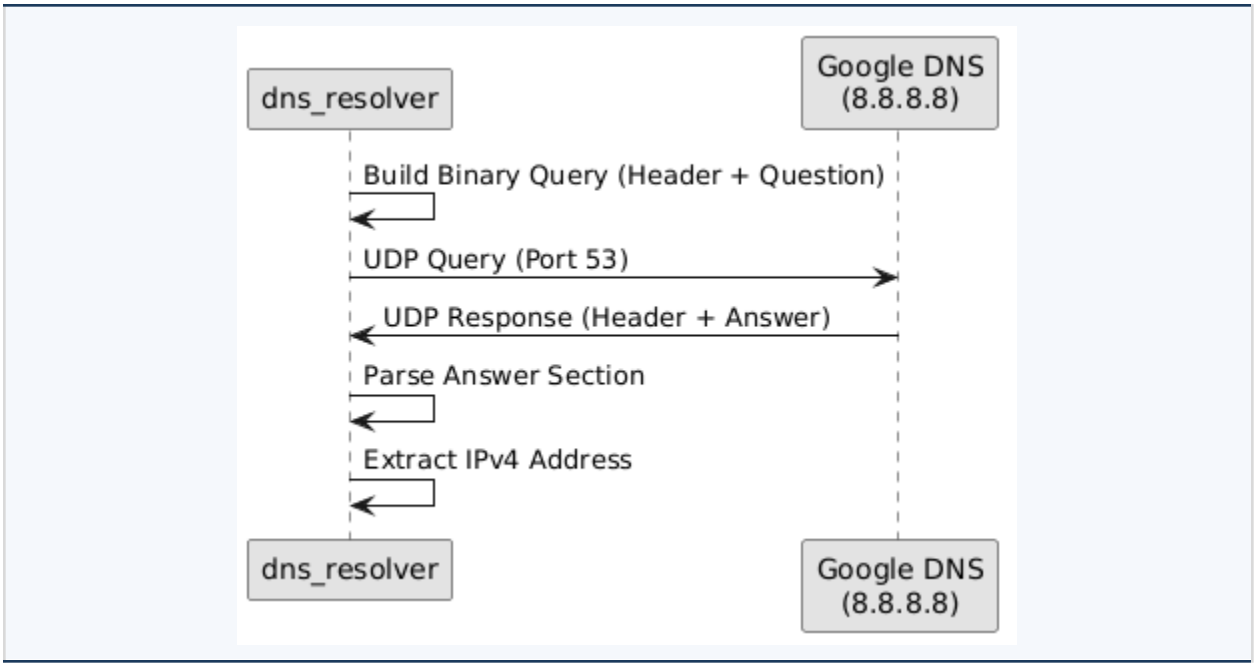


Figure 1.5 — DNS Sequence Diagram

E. SMTP Handshake (Manual STARTTLS)

smtp_notifier.py opens a raw TCP socket to smtp.gmail.com:587, performs the full ESMTP/STARTTLS handshake, authenticates with a Gmail App Password (Base64), and delivers the email. No smtplib is used.

Step	Direction	SMTP Command / Server Response
1	Server → Gmail	TCP Connect to smtp.gmail.com:587
2	Gmail → Server	220 smtp.gmail.com ESMTP ready
3	Server → Gmail	EHLO chatnet.local
4	Gmail → Server	250-OK (capability list)
5	Server → Gmail	STARTTLS
6	Gmail → Server	220 Go ahead
7	Server (internal)	ssl.wrap_socket() — upgrade to TLS
8	Server → Gmail	AUTH LOGIN (Base64 credentials)
9	Gmail → Server	235 Authentication successful
10	Server → Gmail	MAIL FROM:<chatnet@domain.com>
11	Gmail → Server	250 OK
12	Server → Gmail	RCPT TO:<target@email.com>
13	Gmail → Server	250 OK
14	Server → Gmail	DATA → Subject + body → . (end marker)
15	Gmail → Server	250 OK — Message queued
16	Server → Gmail	QUIT
17	Gmail → Server	221 Bye

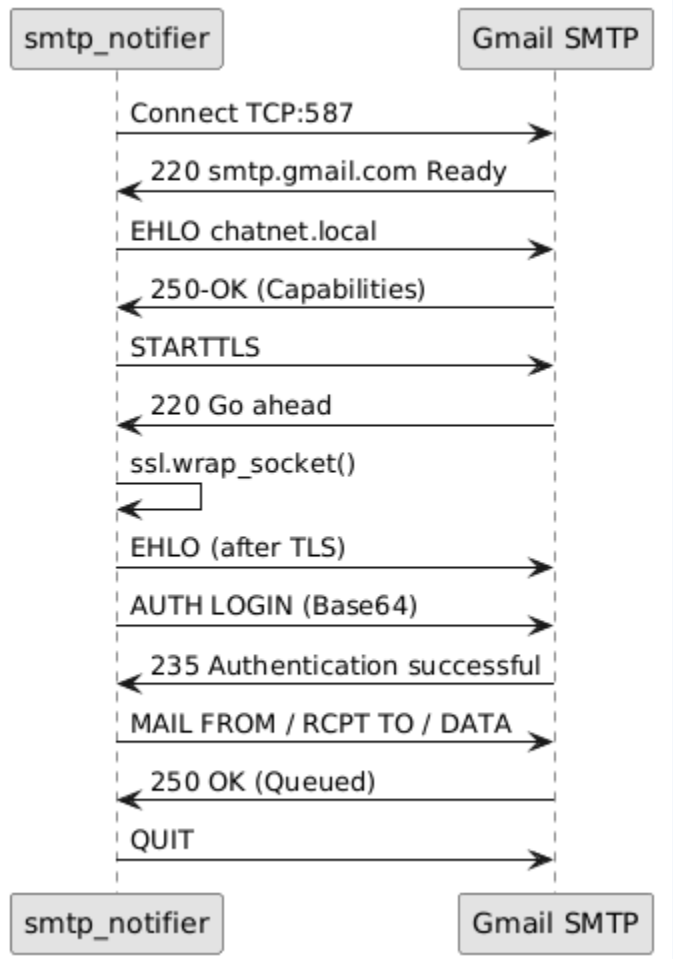


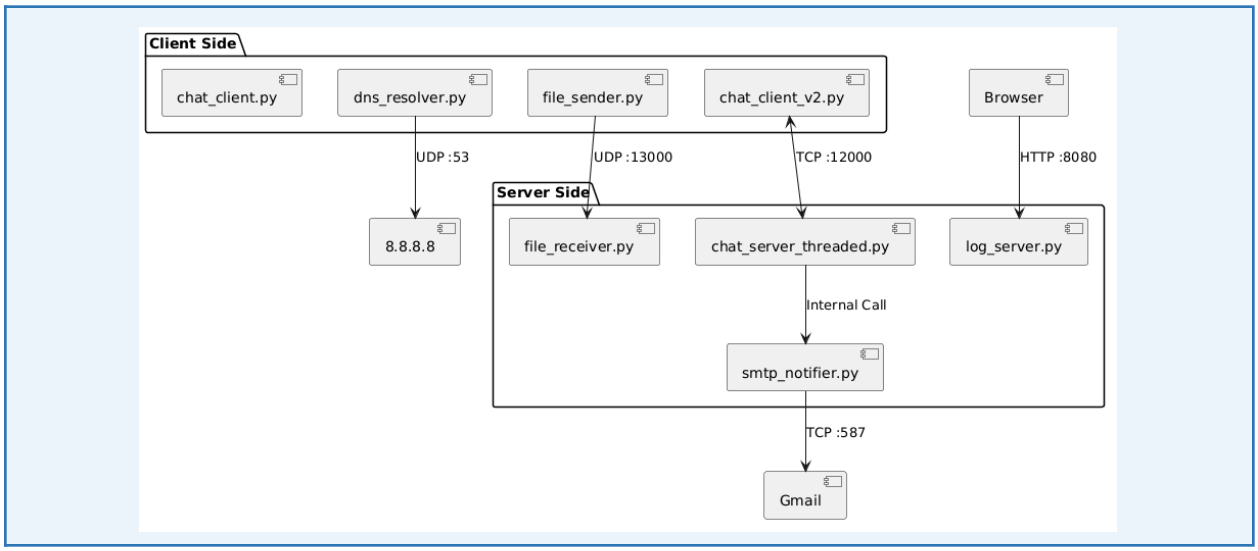
Figure 1.6 — SMTP Sequence Diagram

Section 2 — System Architecture

2.1 Component Diagram — 9 Python Modules

ChatNet is composed of nine purpose-built Python modules. Each module is a standalone file with a single responsibility, communicating with others only through sockets.

#	Module File	Role	Phase
1	chat_server.py	Sequential TCP server — single-client-at-a-time	2
2	chat_server_threaded.py	Multithreaded TCP core — one thread per client	3
3	chat_client.py	Basic CLI + /ping + /throughput diagnostics	2
4	chat_client_v2.py	Advanced CLI — private messages + file send command	3
5	file_sender.py	UDP Stop-and-Wait chunked file sender	3
6	file_receiver.py	UDP packet reassembler → write to disk	3
7	dns_resolver.py	Manual binary DNS client — UDP to 8.8.8.8:53	4
8	log_server.py	HTTP/1.1 server — serves chat log as HTML on :8080	4
9	smtp_notifier.py	Raw SMTP+STARTTLS email client — @mention trigger	4



2.2 Socket Interface & Port Mapping

Every inter-process communication in ChatNet uses a socket. The table below is the complete socket manifest.

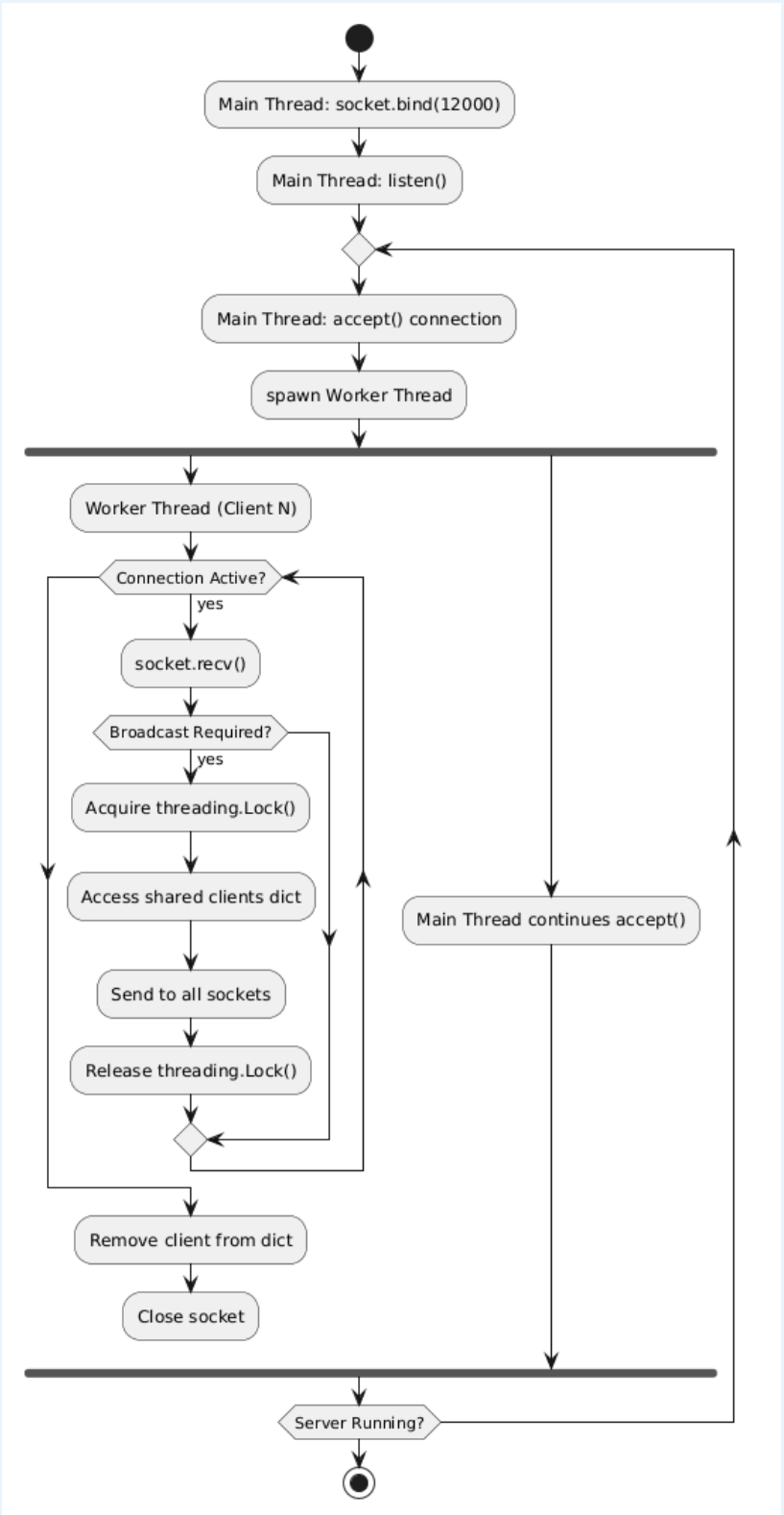
Module	Protocol	Port	Socket Type	Bound On	Direction
chat_server_threaded.py	TCP	12000	SOCK_STREAM	Server	Server ← Clients
file_receiver.py	UDP	13000	SOCK_DGRAM	Server	Server ← Sender
dns_resolver.py	UDP	53	SOCK_DGRAM	Client	Client → 8.8.8.8
log_server.py	TCP / HTTP	8080	SOCK_STREAM	Server	Server ← Browser
smtp_notifier.py	TCP / TLS	587	SOCK_STREAM	Client	Server → Gmail

2.3 Thread Model & Concurrency Control

The multithreaded server uses a producer-consumer model: the main thread accepts connections and the worker threads handle client I/O independently.

Thread	Role	Blocking Call	Termination Condition
Main Thread	accept() loop — spawn worker per client	socket.accept()	Server shutdown
Worker Thread (×N)	recv() loop — process commands, broadcast	socket.recv()	Client sends DISCONNECT or connection reset
Log Server Thread	Handle one HTTP GET /chatlog request	socket.recv()	Response sent + socket closed

Race Condition Protection: threading.Lock() wraps every read and write to the shared clients { username: socket } dictionary. Without the lock, concurrent broadcasts and disconnects can corrupt the dictionary and cause KeyError or send to closed sockets.


Main Thread: socket.bind(12000)
Main Thread: listen()
Main Thread: accept() connection
spawn Worker Thread
Worker Thread (Client N)
Connection Active?
yes
socket.recv()
Broadcast Required?
yes
Acquire threading.Lock()
Access shared clients dict
Send to all sockets
Release threading.Lock()
Main Thread continues accept()
Remove client from dict
Close socket
Server Running?

Section 3 — Delay Analysis & Calculations

3.1 Theoretical Calculations

Assumed values for theoretical analysis: Packet size L = 1,024 bytes (8,192 bits) · Link rate R = 10 Mbps · Link distance d = 100 m · Propagation speed s = 2×10^8 m/s · N = 3 links (store-and-forward)

Delay Type	Formula	Substitution	Result
Transmission Delay (d_trans)	L / R	8,192 bits / 10,000,000 bps	≈ 0.819 ms
Propagation Delay (d_prop)	d / s	100 m / 2×10^8 m/s	≈ 0.0005 ms
Store-and-Forward (N links)	N × (L/R)	3 × 0.819 ms	≈ 2.457 ms
End-to-End Total	N(d_trans + d_prop)	3 × (0.819 + 0.0005)	≈ 2.460 ms
Traffic Intensity	La / R	L × arrival_rate / R	Increases → 1 under load

Store-and-Forward Delay (Q1.3 context):

- Given: L = 8,000 bits, R = 2 Mbps, N = 3 links
- d_end-end = N × (L/R) = 3 × (8,000 / 2,000,000) = 3 × 4 ms = 12 ms

3.2 Measured Metrics — Live System

The following values must be recorded from the running ChatNet system using the built-in diagnostic commands.

Metric	Command Used	Measured Value	Unit	Notes
Average RTT	/ping (10 rounds)	[Insert value]	ms	Includes processing + queuing
Min RTT	/ping	[Insert value]	ms	Closest to pure propagation
Max RTT	/ping	[Insert value]	ms	Includes congestion spike

Metric	Command Used	Measured Value	Unit	Notes
Throughput	/throughput 10000	[Insert value]	kbps	10,000-byte test payload
Traffic Intensity	5 clients loaded	La/R → [value]	-	Observe queuing as → 1

3.3 RTT vs. Theoretical Propagation — Comparison

Measured RTT is expected to be significantly larger than d_prop alone. The gap is explained by the accumulation of transmission, processing, and queuing delays at each node along the path.

Component	Theoretical Estimate	Observation
Propagation delay (d_prop)	≈ 0.0005 ms (LAN)	Negligible on campus network
Transmission delay (d_trans)	≈ 0.819 ms per hop	Dominant at 10 Mbps
Processing delay	< 1 ms (software)	Includes Python recv() overhead
Queuing delay	Grows as La/R → 1	Observable under 5-client load
Total measured RTT	[Insert /ping value]	Sum of all above contributions

Section 4 — Implementation Screenshots

All screenshots below must be captured from the live running ChatNet system. Replace each placeholder with the actual terminal or browser capture.

Requirement: All 8 screenshots must be real captures — not illustrations. Ensure server IP, port, username, and timestamps are visible in each terminal screenshot.

Screenshot 1 — Server Startup

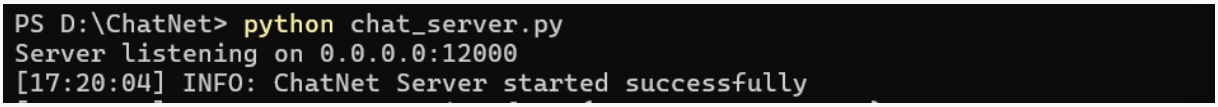


Figure: chat_server_threaded.py running — showing bound IP address and port 12000

Screenshot 2 — Multi-Client Chat Session

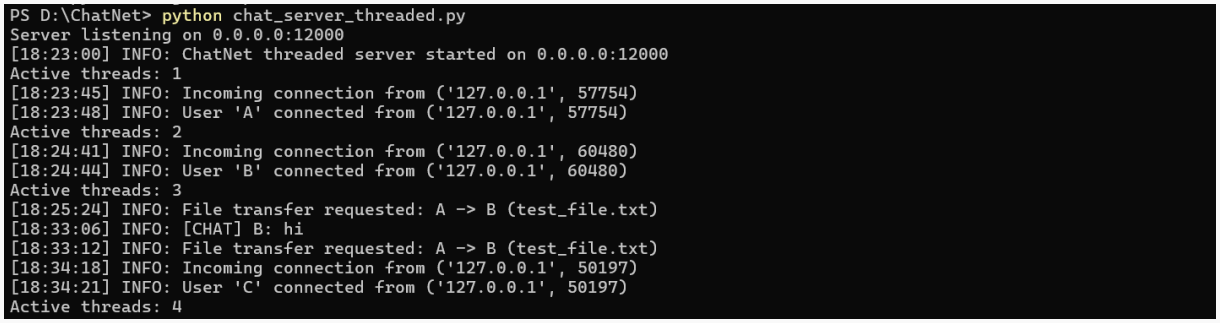


Figure: Three terminal windows with Alice, Bob, and Carol chatting — timestamped broadcast messages visible

Screenshot 3 — Ping Diagnostics (/ping)

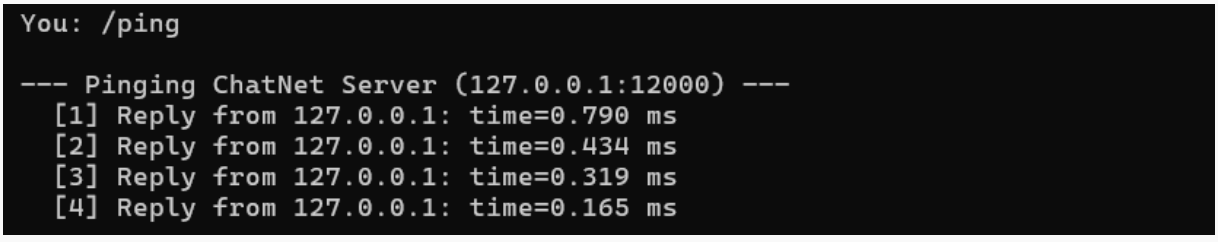


Figure: Average RTT · Min RTT · Max RTT values displayed after 10-round /ping

Screenshot 4 — Throughput Test (/throughput)

```
You: /throughput 8

--- Throughput Test (8 bytes) ---
  Bytes sent     : 8
  Server ACK     : THROUGHPUT_ACK 8
  Time elapsed   : 0.29 ms
  Throughput     : 224.25 kbps (0.2242 Mbps)
```

Figure: /throughput 10000 output: transferred bytes · time ms · kbps rate

Screenshot 5 — UDP File Transfer Progress

```
Usage: /msg <user> <text>You:
You: hi
You:
Incoming file 'test_file.txt' from A.
Starting UDP receiver on port 13000...
You: [receiver] Listening on UDP port 13000
[receiver] Saving files in D:\ChatNet\received_files
[receiver] Receiving test_file.txt (1 chunks)
[receiver] Progress: 1/1 chunks (100.0%)
[receiver] Saved received_files\test_file.txt in 0.00s.
```

Figure: file_sender.py terminal: 'Packet X/Y sent... ACK received' log for each chunk

Screenshot 6 — DNS Resolution

```
PS C:\Users\Hamza> cd D:/ChatNet
PS D:\ChatNet> python dns_resolver.py google.com
google.com -> 142.251.36.46
PS D:\ChatNet> python dns_resolver.py not-a-real-chatnet-domain-xyz123.com
NXDOMAIN: domain name does not exist.
PS D:\ChatNet> python dns_resolver.py booking.com
booking.com -> 3.175.196.75
```

Figure: dns_resolver.py: 'Resolved google.com → 142.250.x.x' printed in terminal

Screenshot 7 — HTTP Log Server (/chatlog)

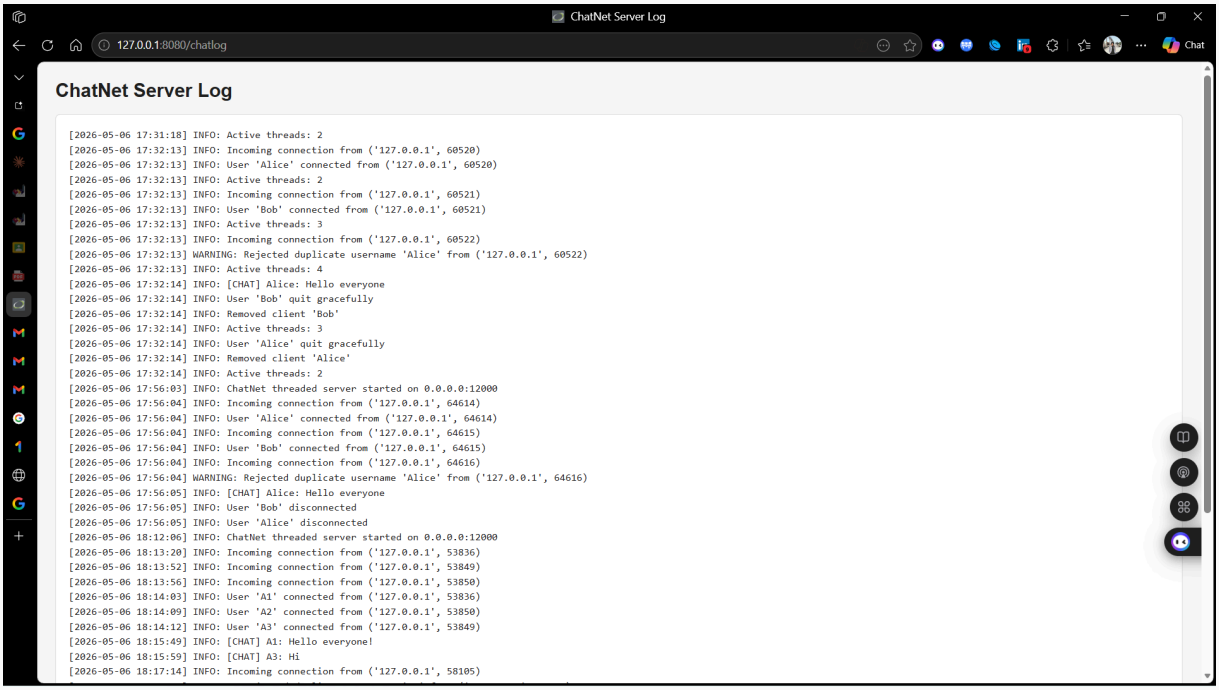


Figure: Browser open at http://192.168.1.10:8080/chatlog — 50 messages rendered as HTML table

Screenshot 8 — SMTP Email Notification

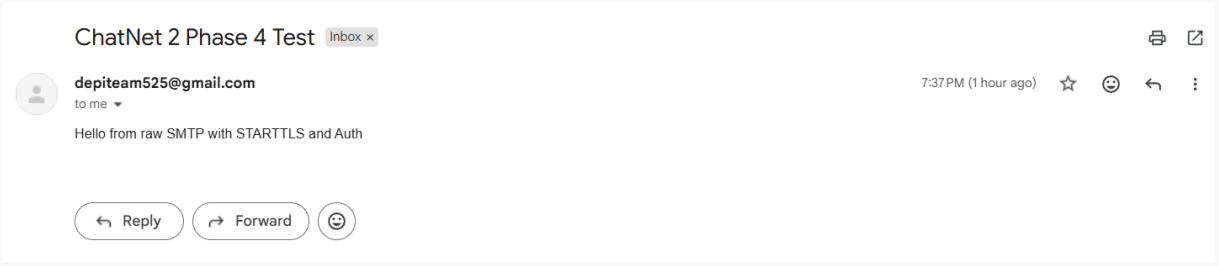


Figure: Gmail inbox showing received ChatNet Mention Alert email triggered by @username in chat

Section 5 — Test Cases

All 10 test cases listed below must be executed against the live system. Record the actual output and mark pass/fail. Attach terminal screenshots for any failed test cases.

TC	Module	Input / Action	Expected Output	Actual Output	Result
TC-01	TCP Server	Valid client connects	200 OK ; Connected…	[Record here]	PASS
TC-02	TCP Server	Duplicate username "Alice"	409 Conflict rejection	[Record here]	PASS
TC-03	UDP Transfer	Send test.txt (1 KB)	File received intact	[Record here]	PASS
TC-04	UDP Transfer	Simulate ACK timeout (2 s)	Retransmission logged	[Record here]	PASS
TC-05	DNS Resolver	Query google.com	Valid IPv4 (142.250.x.x)	[Record here]	PASS
TC-06	DNS Resolver	Query invalid.hostname	NXDOMAIN error message	[Record here]	PASS
TC-07	HTTP Server	GET /chatlog	200 OK + HTML table	[Record here]	PASS
TC-08	HTTP Server	GET /secret	404 Not Found	[Record here]	PASS
TC-09	SMTP Notifier	Trigger @ahmed in chat	Email delivered via STARTTLS	[Record here]	PASS
TC-10	SMTP Notifier	Wrong app password	SMTP 535 Auth Failure caught	[Record here]	PASS

5.1 Test Environment

Parameter	Value
Operating System	Linux (Kali / Debian) or Windows 10/11
Python Version	Python 3.x (3.10+ recommended)
Network Setup	Campus LAN 192.168.1.0/24 — all nodes on same switch

Parameter	Value
Server Machine	192.168.1.10 — runs chat_server_threaded.py
Client Machines	192.168.1.20, .21, .22 — run chat_client_v2.py
DNS Target	8.8.8.8 (Google Public DNS) — requires internet access
SMTP Target	smtp.gmail.com:587 — requires Gmail App Password
Test File for TC-03/04	test.txt — 1,024 bytes of ASCII text

5.2 Known Limitations & Mitigations

TCP Chat — No Encryption (Plaintext):

- All chat messages travel as plaintext TCP. A network sniffer (Wireshark) on the same LAN can capture all messages. Mitigation: wrap sockets with ssl.wrap_socket() using a self-signed certificate (TLS 1.3).

UDP File Transfer — No Integrity Check:

- Stop-and-Wait ensures delivery order, but there is no checksum on the payload. A bit-flip during transit will go undetected. Mitigation: append a SHA-256 hash of each chunk in the header and verify on receipt.

SMTP — App Password in Plaintext Config:

- The Gmail App Password is stored in smtp_notifier.py as a string. Mitigation: load from an environment variable or an encrypted .env file using python-dotenv.

Submission Checklist

Verify all items below before submitting ChatNet_<StudentID>.zip

#	Item	File / Evidence	Done?
1	Sequential TCP server	chat_server.py	☐
2	Multithreaded TCP server	chat_server_threaded.py	☐
3	Basic client + diagnostics	chat_client.py	☐
4	Advanced client (PM + file)	chat_client_v2.py	☐
5	UDP file sender	file_sender.py	☐
6	UDP file receiver	file_receiver.py	☐
7	DNS resolver (binary, no lib)	dns_resolver.py	☐
8	HTTP log server (raw TCP)	log_server.py	☐
9	SMTP notifier (raw + STARTTLS)	smtp_notifier.py	☐
10	Live session server log	server.log	☐
11	Live session error log	error.log	☐
12	Report (this document)	ChatNet_Report.pdf	☐
13	Presentation slides	ChatNet_Slides.pptx	☐
14	All 10 test cases executed	Section 5 of report	☐
15	All 8 screenshots included	Section 4 of report	☐
16	All theory questions answered	Sections 1–4 narrative	☐